

CHECKPOINT 04

Containers na nuvem

Containerizacao de um portal estatico, publicacao no Docker Hub e execucao em uma instancia Amazon EC2.

Aluno: Guilherme Silva dos Santos

RM: 551168

Turma: 4ESPX

Disciplina: Engenharia de Software

Data: 28/08/2026

Sumario

Este documento registra a implementacao e a validacao do projeto CP04. O portal foi containerizado com Nginx, publicado com a tag **cp04** no Docker Hub e executado em uma EC2 publica na regioao **us-east-1**.

Secao	Conteudo
1. Objetivo	Contexto e escopo do projeto.
2. Portal personalizado	Identificacao e fundamentos de containers.
3. Docker	Build, imagem e container saudavel.
4. Docker Hub	Repositorio publico e tag cp04.
5. Infraestrutura AWS	VPC, sub-rede, gateway, rotas e Security Group.
6. EC2 e validacao	Instancia, IP publico, URL e Terraform.
7. Conclusao	Resultado e reprodutibilidade.

Resultado geral: execucao concluida com sucesso. A imagem foi construida e publicada, 7 recursos AWS foram criados pelo Terraform e o portal respondeu pela URL publica.

Objetivo

A Arcanjo SA definiu que novas aplicações devem ser executadas em containers para reduzir diferenças entre ambientes e tornar o deploy reproduzível. O projeto atende esse piloto por meio de um portal estático, uma imagem Docker pública e uma infraestrutura AWS declarada em Terraform.

O fluxo validado foi: HTML/CSS -> imagem Docker -> Docker Hub -> EC2 Amazon Linux 2023 -> acesso HTTP público.

Portal personalizado

O portal apresenta a identificacao do estudante e uma explicacao visual sobre os mecanismos que sustentam o isolamento e o controle de recursos dos containers.

Item	Conteudo validado
Aluno	Guilherme Silva dos Santos
RM	551168
Turma	4ESPX
Conteudo tecnico	Namespaces, cgroups, isolamento, CPU, memoria, I/O e processos.
URL publica	http://3.85.240.135

Namespaces definem o que cada processo consegue enxergar: processos, rede, sistema de arquivos, hostname, comunicacao e usuarios. cgroups definem quanto cada grupo de processos pode consumir, permitindo limitar e acompanhar CPU, memoria, disco/I/O e quantidade de processos.

Em conjunto, esses recursos permitem compartilhar o kernel do host mantendo isolamento e controle. O Docker organiza essas capacidades e adiciona imagens, redes e uma experiencia padronizada de execucao.

Evidencia visual: portal aberto no endereco publico, com a identificacao do aluno e as secoes Namespaces e cgroups legiveis.

Construcao e execucao com Docker

O Dockerfile utiliza a imagem oficial **nginx:alpine**, copia o portal para o diretorio padrao do Nginx, expoe a porta 80 e define um healthcheck HTTP.

```
docker build -t guilhermessilva2004/cp04-site:cp04 .
[+] Building 3.7s (7/7) FINISHED
=> naming to docker.io/guilhermessilva2004/cp04-site:cp04
```

REPOSITORY	TAG	IMAGE ID	SIZE
guilhermessilva2004/cp04-site	cp04	8150f7efe565	62.8MB

CONTAINER ID	IMAGE	STATUS	PORTS
5940e348e996	guilhermessilva2004/cp04-site:cp04	Up (healthy)	0.0.0.0:8080->80/tcp

A construcao terminou sem erro. O container local respondeu em HTTP e foi reportado como **healthy**, comprovando que a imagem publicada funciona antes da execucao na nuvem.

Publicacao no Docker Hub

A imagem foi publicada em um repositorio publico do Docker Hub, com a tag exigida pelo enunciado.

Item	Valor
Repositorio	guilhermessilva2004/cp04-site
Visibilidade	Public
Tag	cp04
Imagem completa	guilhermessilva2004/cp04-site:cp04
Digest	sha256:79cd4905b532da7bf7d7d309a69c171914e90f4423688c6450481a8427091672
Link	Docker Hub - tags

A EC2 utiliza exatamente a mesma referencia publica, reduzindo a diferenca entre o ambiente de teste e o ambiente de execucao.

Infraestrutura AWS

A infraestrutura foi criada com Terraform na região us-east-1. A configuração contempla rede pública, acesso HTTP e uma instância preparada pelo user_data para instalar o Docker e executar a imagem publicada.

Recurso	Configuração validada	Identificação
VPC	CIDR 10.0.0.0/24; estado available	vpc-05cf759ee202832c5
Sub-rede pública	CIDR 10.0.0.0/26; PublicIPv4=True; us-east-1d	subnet-026c681c9ccff30a6
Internet Gateway	Conectado a VPC; estado available	igw-02478c97747fd4a00
Tabela de rotas	0.0.0.0/0 -> Internet Gateway; active	rtb-0330575a29e6cd653
Associação	Sub-rede pública associada; Main=False	rtbassoc-0111f089d0cb170bf
Security Group	TCP 80 de 0.0.0.0/0; saída liberada	sg-0ae5c3fa7ac6d2a85

A rota default permite o tráfego de saída e o Security Group autoriza o acesso HTTP necessário ao portal. SSH permaneceu fechado por padrão.

EC2 e validacao da execucao

Item	Resultado
Instancia	i-0c0129d1fa1b7f999
Tipo	t2.micro
Estado	running
IP privado	10.0.0.41
IP publico	3.85.240.135
Imagem executada	guilhermessilva2004/cp04-site:cp04
URL publica	http://3.85.240.135

O bootstrap da instancia instalou o Docker, baixou a imagem publica e iniciou o container na porta 80. A validacao externa retornou HTTP 200 e encontrou o nome do estudante no conteudo servido.

```
Apply complete! Resources: 7 added, 0 changed, 0 destroyed.
```

```
docker_image = "guilhermessilva2004/cp04-site:cp04"
instance_id   = "i-0c0129d1fa1b7f999"
public_ip     = "3.85.240.135"
site_url      = "http://3.85.240.135"
```

```
SUCESSO: portal disponivel em http://3.85.240.135
```

Atencao: o IP publico e temporario e pode mudar se a instancia for recriada. Registre as evidencias antes de destruir os recursos.

Links e conclusao

Recurso	Endereco
GitHub	github.com/GuilhermeSSantos2004/cp04-es-docker-aws
Docker Hub	hub.docker.com/r/guilhermessilva2004/cp04-site/tags
Portal EC2	http://3.85.240.135

Conclusao

A containerizacao transformou o portal e suas dependencias em uma unidade portatil, validada localmente e reutilizada na EC2. A infraestrutura declarativa criou uma rede publica reproduzivel, com rota para o Internet Gateway, controle de trafego no Security Group e uma instancia preparada automaticamente para executar a imagem do Docker Hub.

O resultado atende ao objetivo do CP04: uma entrega consistente, documentada e acessivel por uma URL publica, sem depender de instalacao manual do Docker na instancia.

Observacao de seguranca: este relatorio nao inclui access keys, session tokens, senhas, tokens do Docker Hub, ARN ou ID da conta AWS.